

Gestor e Biblioteca de Música: relatório técnico

Ricardo Monteiro (1240842), João Henrique (1201253), Alexei Godea (1241115)

Resumo: O presente relatório técnico serve como guia para entendimento e esclarecimento do trabalho, e motivação do mesmo, realizado ao longo do desenvolvimento do programa de Gestor e Biblioteca de música, e, ao mesmo tempo, explicita o desenho e funcionamento do mesmo.

Palavras-chave: engenharia de software, análise, desenho, implementação, testagem, C++, programação

1 Introdução

No contexto do projeto proposto na unidade curricular de Fundamentos de Desenvolvimento de Software, foi concebida a ideia do programa Gestor e Biblioteca de Música, que tem como objetivo proporcionar um programa de linha de comandos simples, intuitivo e robusto, com funcionalidades que permitirão ao utilizador gerir a sua biblioteca pessoal de música como pretender, bem como oferecer a possibilidade de partilhar músicas com outros utilizadores na mesma máquina.

Na primeira iteração, que consistiu na análise de software, foram dados os primeiros passos essenciais, incluindo: a criação do modelo de domínio, lista de funcionalidades, casos de uso, diagramas de sequência e glossário.

Na sequência do trabalho realizado na primeira iteração, dedicámo-nos ao desenho do nosso programa e começámos também a fase da implementação.

Concluindo, na terceira e última iteração, realizou-se a implementação completa e a testagem do nosso programa, bem como a escrita do presente relatório.

2 Entendimento do problema

Com vista a proporcionar um programa de linha de comandos simples, intuitivo e robusto, com funcionalidades que permitirão ao utilizador gerir a sua biblioteca pessoal de música da maneira como pretender, bem como oferecer a possibilidade de partilhar músicas com outros utilizadores na mesma máquina, assim como reproduzir músicas, realizou-se o estudo do problema, que resultou na concepção do modelo de domínio, lista de funcionalidades, casos de uso, diagramas de sequência e glossário.

3 Análise

A fase de análise, também chamada de análise de requisitos, é a etapa em que se procura entender profundamente o problema a resolver antes de começar a desenhar a solução.

Implica também a constituição de uma lista de casos de uso, requisitos funcionais e não funcionais.

Começamos pela criação do modelo de domínio e definição dos casos de uso e requisitos.

3.1 Modelo de domínio

O modelo de domínio representa graficamente os elementos principais do sistema, assim como as relações entre eles. Para o programa que é pretendido construir, definimos as entidades e relações coerentes entre elas, tendo assim uma boa fundação para o resto do modelo do programa.

O modelo de domínio apresenta-se na **Error! Reference source not found.** a seguir.

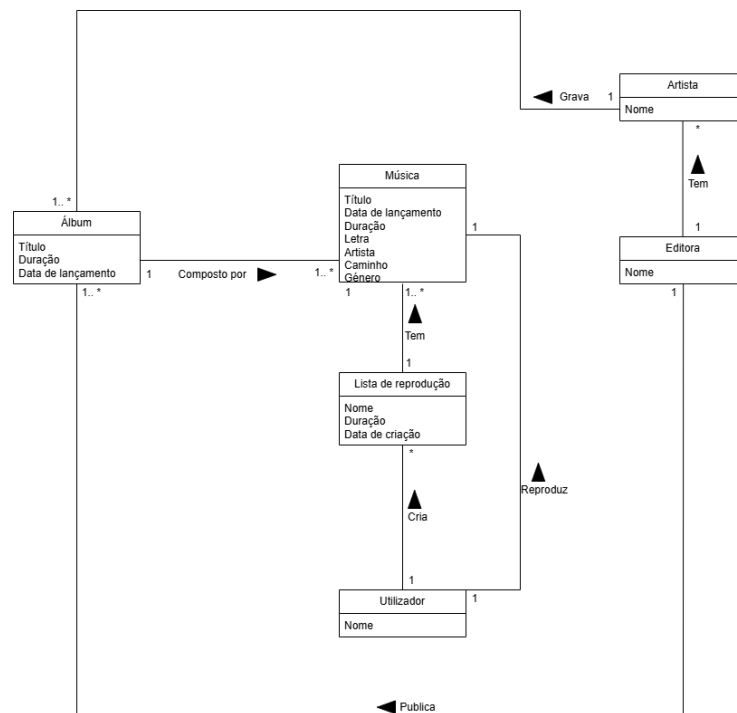


figura 1

3.2 Requisitos

Os casos de uso (UCs) definem as funcionalidades que o programa deve desempenhar. Eles permitem-nos, em sequência, definir os requisitos funcionais e não funcionais. Depois de definir os casos de uso, compomos o diagrama apresentado a seguir:



figura 2

De seguida, definimos os requisitos e desenhámos os SSD (system sequence diagrams). É de notar que alguns casos de uso têm especificações e SSD extremamente parecidos, pelo que, para facilitar a leitura do relatório e a compreensão da estrutura do programa, esses casos de uso serão omitidos. Assim, apresentamos a seguir apenas os seguintes casos de uso: Adicionar Música à Lista de Reprodução, Adicionar Música, Criar Conta, Iniciar Sessão, Partilhar Música, Pesquisar Música, Remover Música da Lista de Reprodução, Remover Música, Reproduzir Música, Sair da Conta, Ver Músicas e Ver Partilha.

SSD – Criar Conta

Na figura 3 está apresentada a forma de como o utilizador pode criar uma conta no programa, e os passos que o programa deve fazer para haver um funcionamento correto da criação da conta.

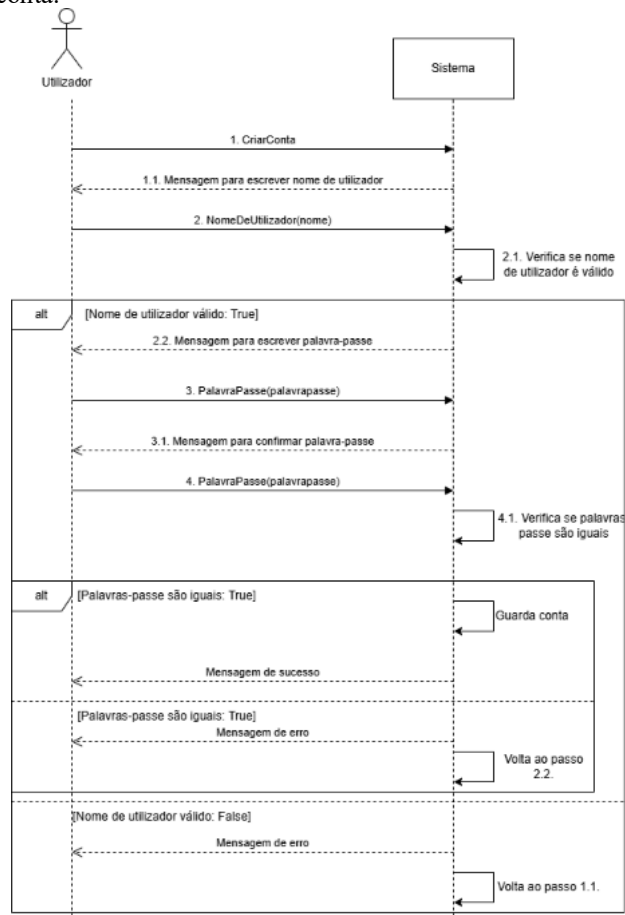


figura 3

SSD – Iniciar Sessão

O sistema também verifica o início de sessão através do processo mostrado na figura 4.

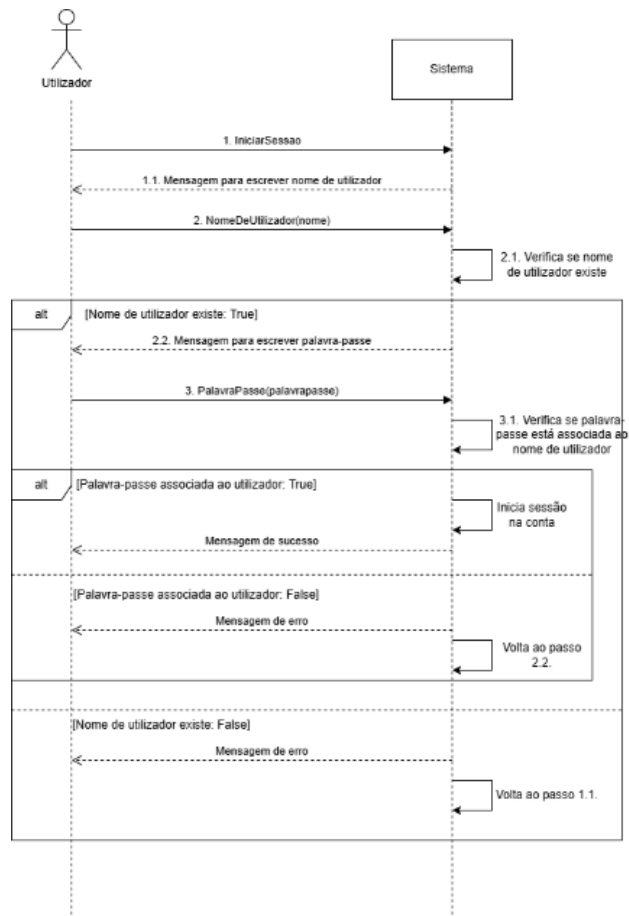


figura 4

UC – Adicionar Música

Através da introdução dos atributos da música, sendo esses o seu título, duração, ano de lançamento, nome do artista, letra, género e caminho do ficheiro onde a música se encontra no computador, para se poder aceder ao ficheiro e tocar a música através do programa. O mesmo fluxo que se encontra na figura 5 é semelhante para artistas, editoras, listas de reprodução e álbuns, sendo que a principal mudança são os atributos.

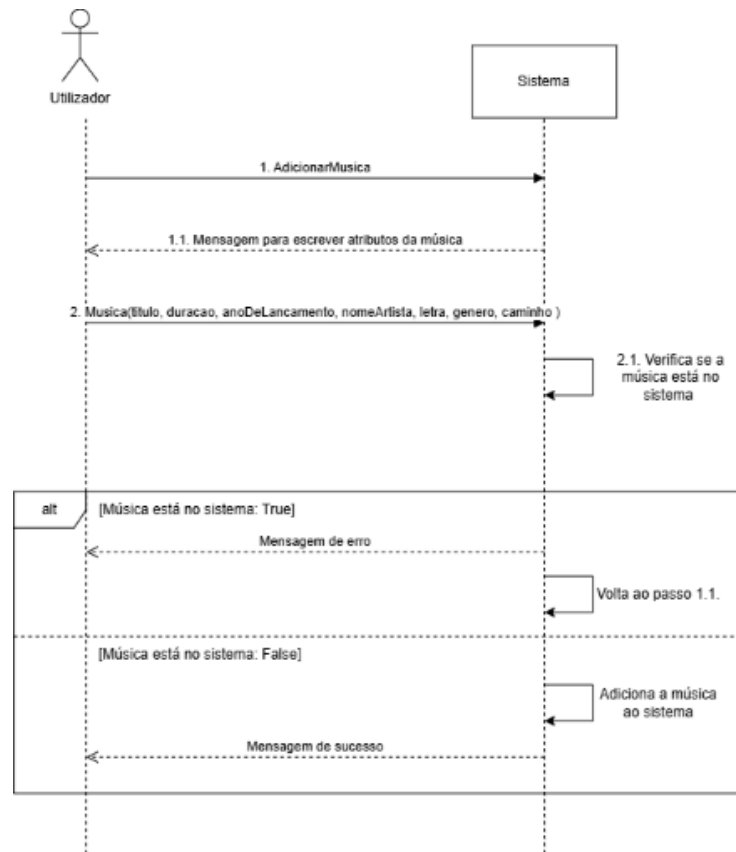


figura 5

UC – Remover Música

Da mesma forma onde se podem adicionar músicas, também se podem remover. O mesmo é aplicável aos mesmos elementos referidos na UC anterior e estão representados na figura 6.

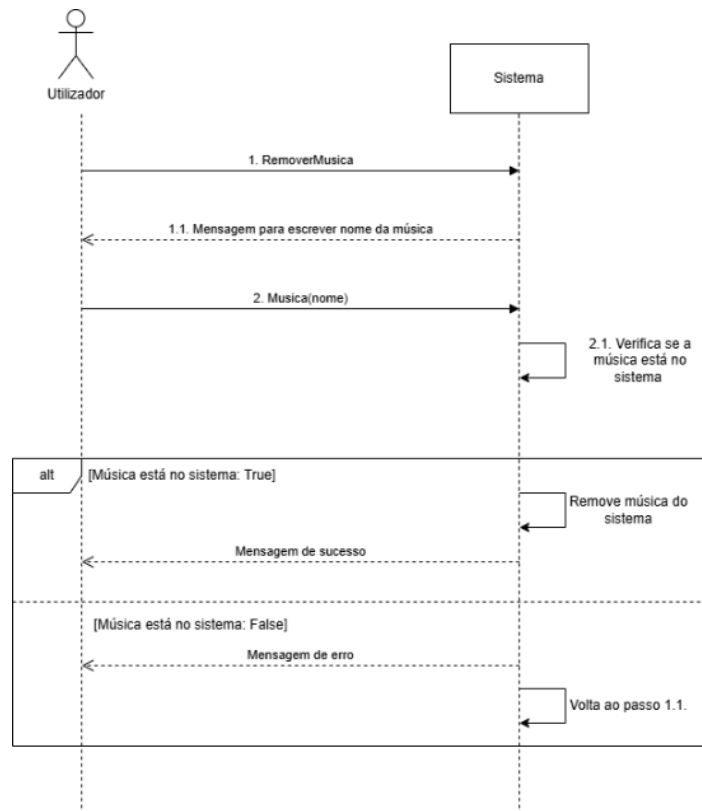


figura 6

UC – Adicionar Música à Lista de Reprodução

É possível fazer relação de elementos, sendo esses a relação de álbuns com artistas, músicas com álbuns, músicas com listas de reprodução, e artistas com editoras. Como exemplo, presente na figura 7, mostramos como se adiciona uma música a uma lista de reprodução.

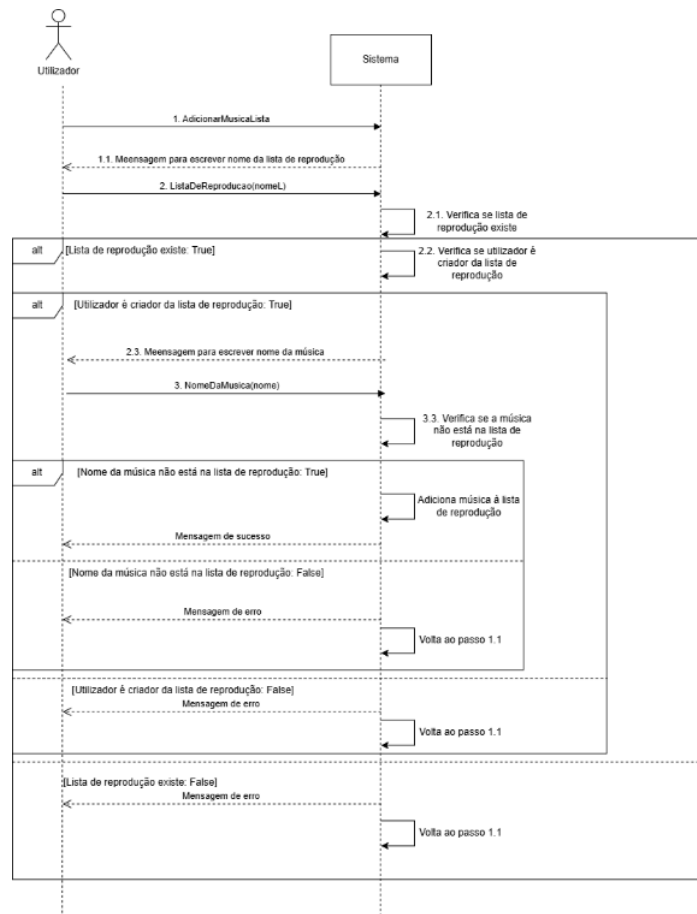


figura 7

UC – Remover Música da Lista de Reprodução

Da mesma forma como se podem adicionar músicas, editoras, artistas, álbuns e listas de reprodução, e fazer a relação de uns entre outros, também podemos fazer a desassociação dos mesmos. Presente na figura 8 está a forma de como se remove uma música da lista de reprodução.

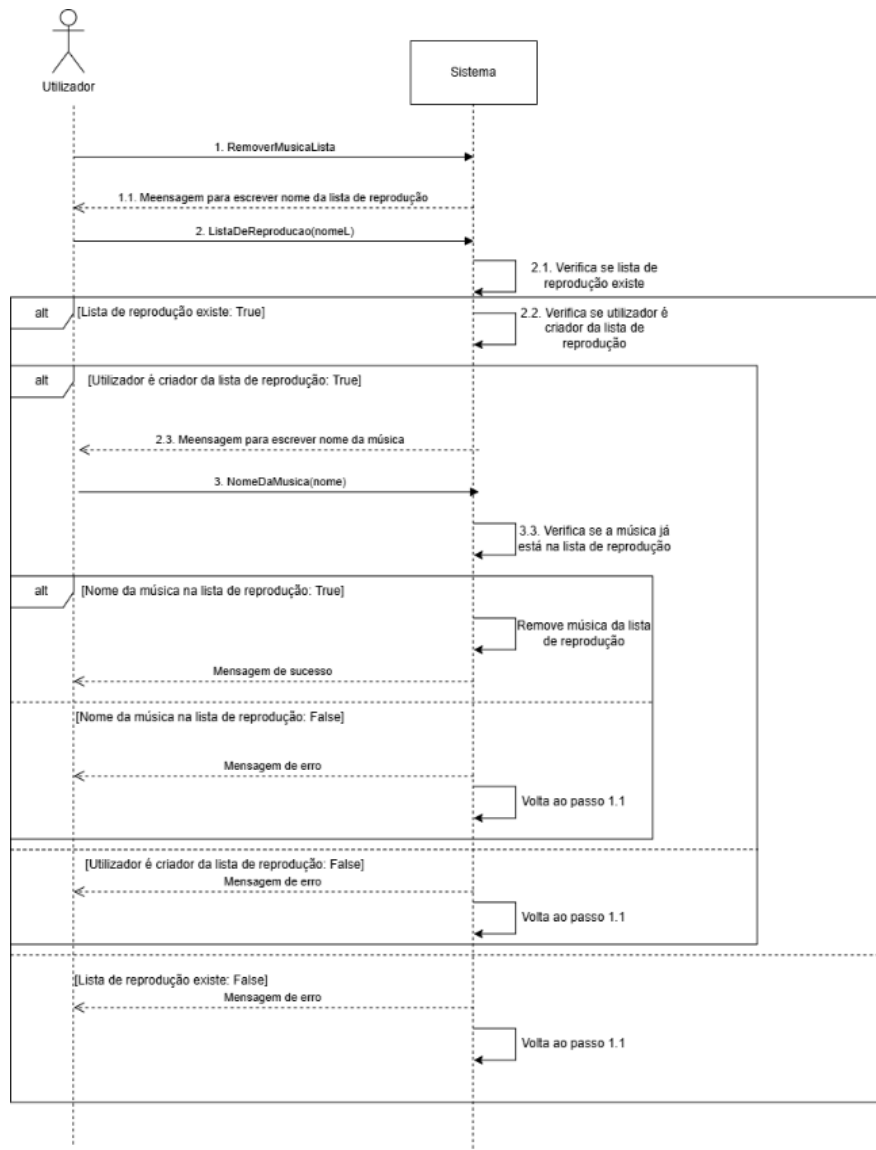


figura 8

UC – Pesquisar Álbum

Para fazer pesquisas, usamos o exemplo de pesquisa de album, na figura 9.

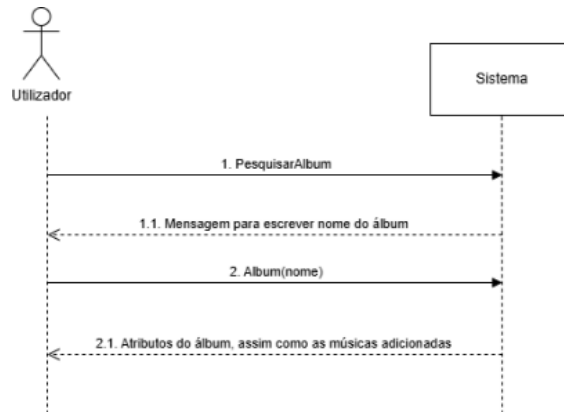


figura 9

UC – Ver Músicas

Para o utilizador ter ideia das músicas que existem no programa, pode vê-las, através do processo presente na figura 10. Isto também é aplicável às UCs de ver editoras, álbuns, listas de reprodução e artistas.



figura 10

UC – Ver Partilha

Para além de ver elementos, o utilizador pode também ver listas de reprodução e músicas que foram partilhadas com ele, na figura 11.

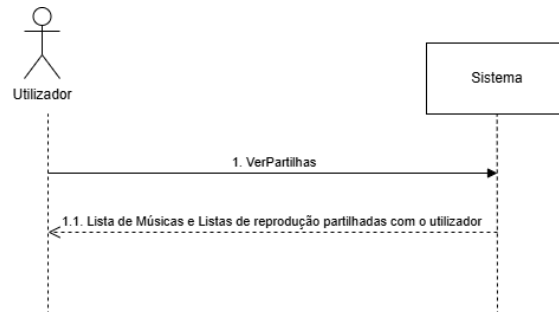


figura 11

UC – Partilhar Música

Para ver as partilhas, os utilizadores também devem ter a opção de partilhar músicas e listas de reprodução. O exemplo de como partilhar uma música está na figura 12.

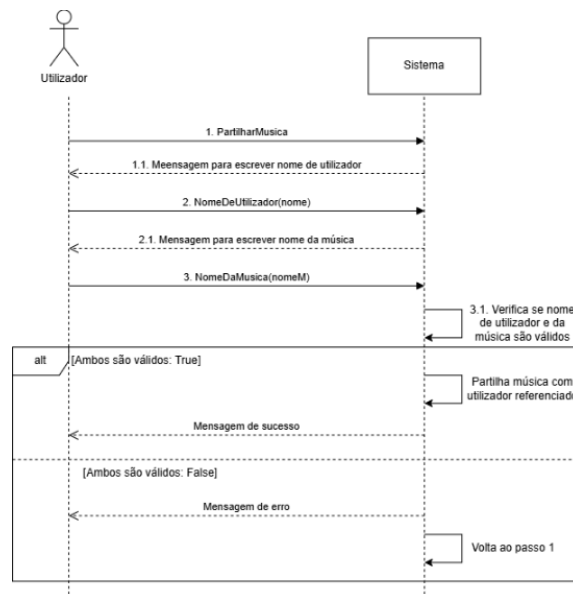


figura 12

UC – Reproduzir Música

Acedendo ao ficheiro de música, o utilizador consegue escolher a opção de reproduzir música.

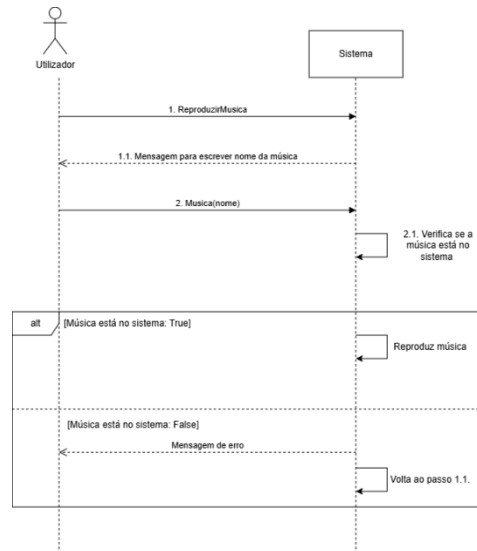


figura 13

UC – Saír da conta

Por fim, o utilizador pode saír da conta, onde dá espaço a outros utilizadores que iniciem sessão e possam utilizar o programa também.

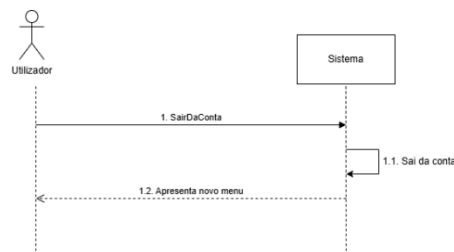


figura 14

3.3 Glossário

Para haver um melhor entendimento dos elementos, o glossário apresentado a seguir permite dar uma breve mas clara explicação dos termos.

Termo técnico	Definição
Álbum	Coleção de várias músicas lançadas em conjunto por um artista (ou banda), geralmente com um título e uma capa.
Artista	Criador ou intérprete da música. Pode ser um cantor individual, uma banda, um compositor ou um produtor.
Editora	Empresa responsável pela produção, distribuição e promoção da música e dos artistas.
Género	Categoria que classifica a música com base no seu estilo, ritmo, instrumentação ou até contexto cultural.
Lista de reprodução	Seleção personalizada de músicas agrupadas pelo utilizador ou pela plataforma.
Música	Unidade básica da plataforma. Um ficheiro de áudio individual que pode ser reproduzido, adicionado a listas de reprodução, etc.
Utilizador	Pessoa que interage com a aplicação para ouvir músicas, criar listas de reprodução e muito mais.

4 Desenho

A fase de desenho é onde se define como o sistema vai ser construído, transformando os requisitos levantados na análise numa arquitetura e especificação técnica detalhada. É a ponte entre "o que fazer" e "como fazer". Começamos, portanto, por definir a arquitetura do sistema.

4.1 Arquitetura do sistema

A arquitetura do sistema descreve a estrutura geral da aplicação, evidenciando a organização da aplicação e como os componentes interagem entre si.

Através da arquitetura física, representada na figura 15, a arquitetura lógica, representada na figura 16, e a organização do código, na figura 17, a organização fica evidenciada de forma completa.

4.2 Arquitetura Física

O diagrama de implementação presente na figura 15 evidencia que o programa é uma aplicação independente que corre localmente na máquina do utilizador.

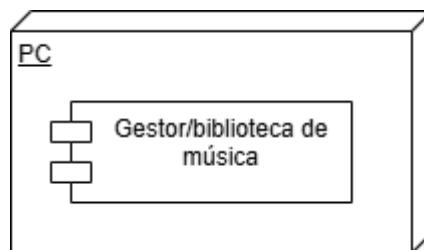


figura 15

4.3 Arquitetura lógica

A arquitetura lógica, representada através do diagrama de componentes, na figura 16, destaca os diferentes módulos e as relações entre os componentes da aplicação.

O Gestor e Biblioteca de Música utiliza o padrão arquitetural Model-View-Controller (MVC), que divide a aplicação em três componentes principais. O Model gere os dados e o estado da aplicação, sem qualquer preocupação com a interface ou apresentação visual, que é gerida pelo View. O Controller gere a ligação entre os dois.

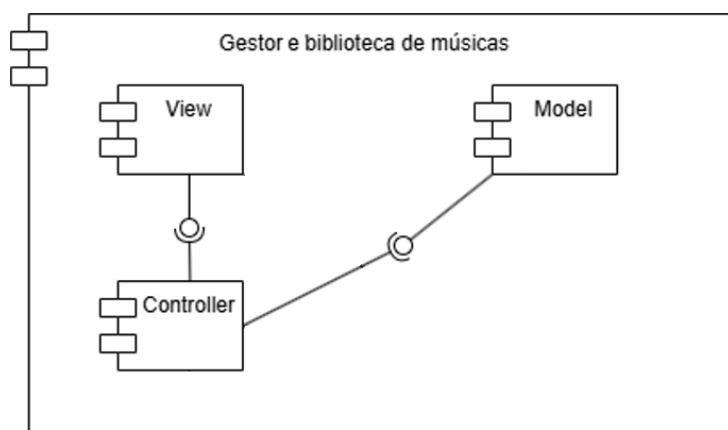


figura 16

4.4 Organização de código

Como está evidenciado na figura 17, o main.cpp é utilizado para iniciar o processo de execução do programa. O main.cpp contém “Controllers”, onde está localizada a parte principal do programa. No controlador estão encontradas todas as funções definidas nas outras partes do programa (views, model, e repo).

Sendo que nos views estão contidos ficheiros que fazem o trabalho de recolha e amostragem de dados, no model estão contidos ficheiros que fazem validação e tratamento desses mesmos dados, e no repo está contido o ficheiro repositório, onde estão contidas as funções que permitem a persistência dos dados, contidos no pacote de dados. O model também acessa aos dados para aceder a ficheiros de música.

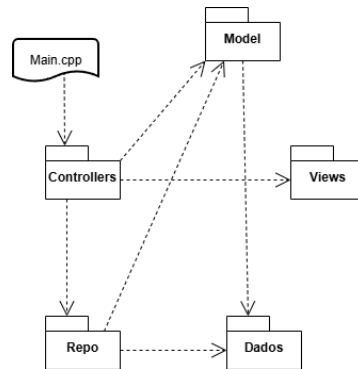


figura 17

4.4 Diagrama de classes de modelo

No presente diagrama de classes de modelo, apresentado na figura 18, apresentamos a estrutura do modelo do nosso programa.

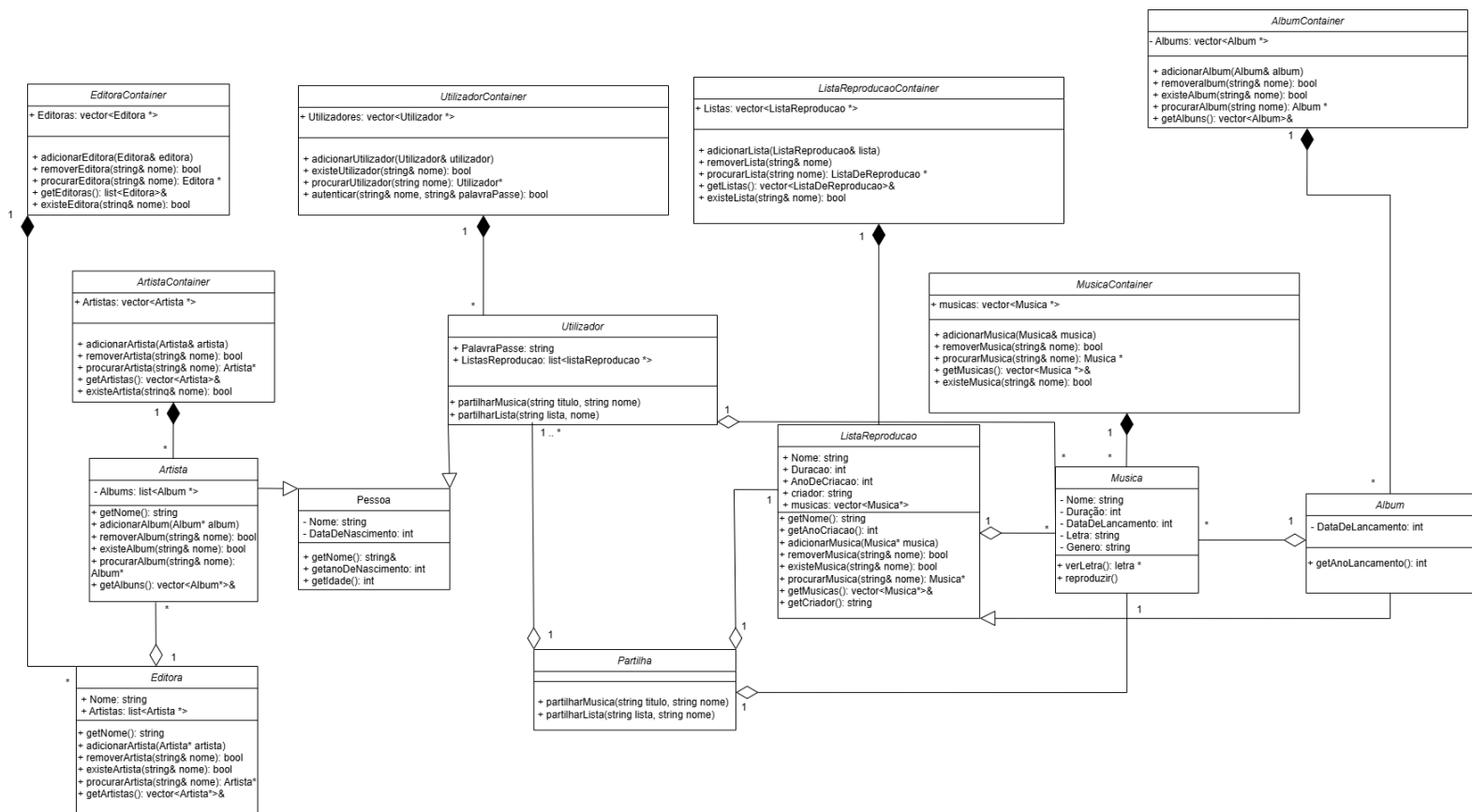


figura 18

4.5 Diagrama de Classe do Controller

O Controller serve como intermediário entre a interface do utilizador (View) e os processos lógicos do programa, que estão no model. Na figura 19 observa-se o Controller e as suas relações com as outras classes.

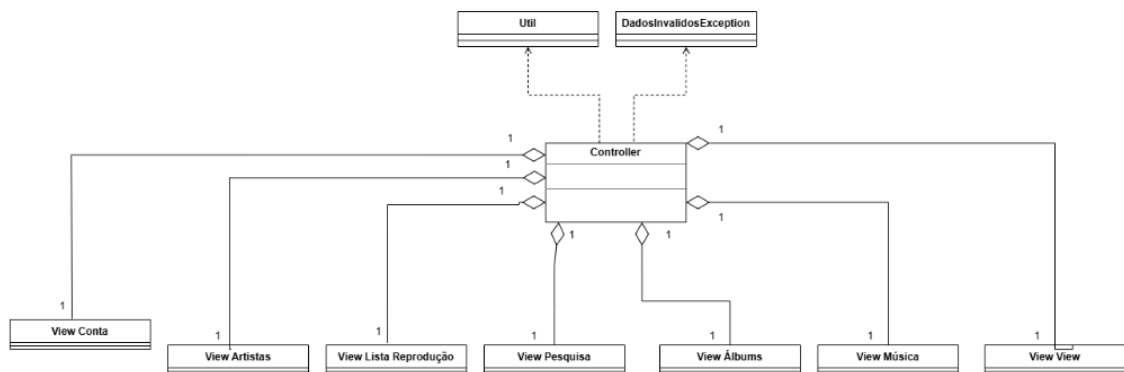


figura 19

4.6 Diagrama de Classes das Views

O diagrama de classes das views apresentado na figura 20, delinea as classes responsáveis pela interação do utilizador com o programa, permitindo assim a apresentação de menus e recolha de dados introduzidos na aplicação.

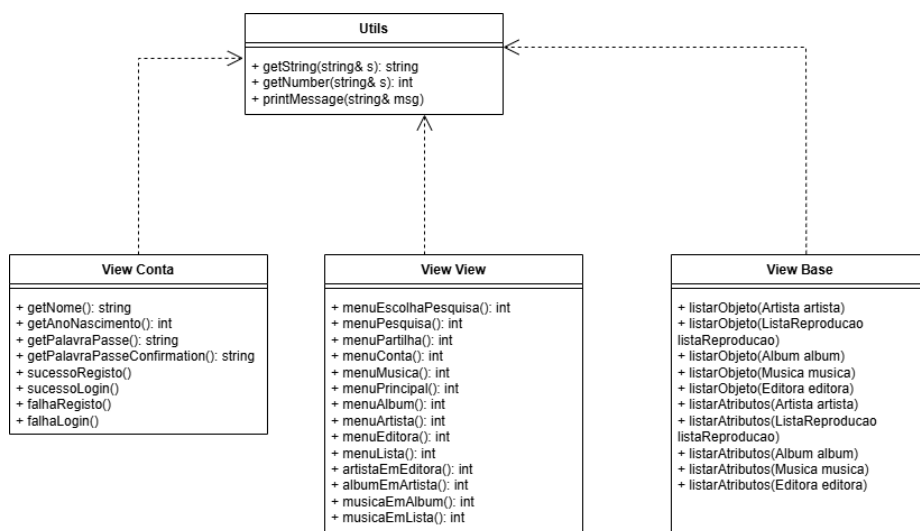


figura 20

4.7 Diagramas de Sequência

Os diagramas de sequência representam a interação entre os componentes do sistema durante a utilização do programa, para os casos de utilização. Demonstram o que deve acontecer entre objetos ao longo da execução.

UC – Criar Conta

O diagrama apresentado na **Error! Reference source not found.** representa o fluxo de execução da criação de conta, demonstrando a interação entre os diferentes componentes envolvidos no processo de registo de um utilizador.

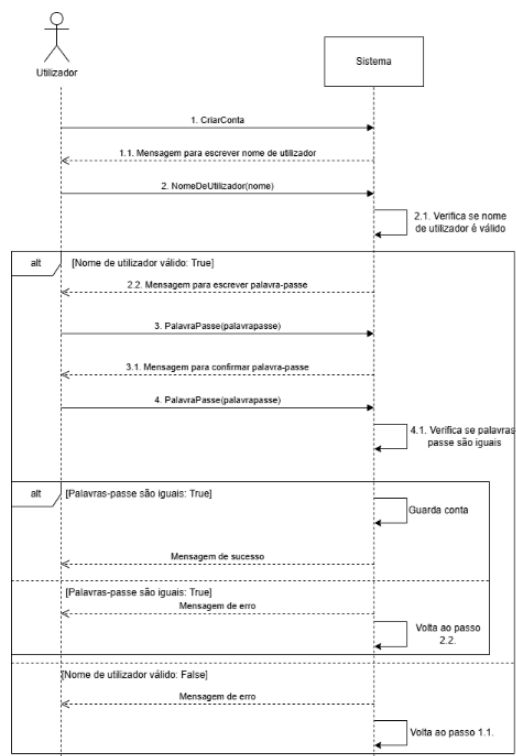


figura 21

UC – Iniciar Sessão

O diagrama apresentado na figura 22 representa o processo de autenticação do utilizador, demonstrando a validação das credenciais e o acesso ao sistema.

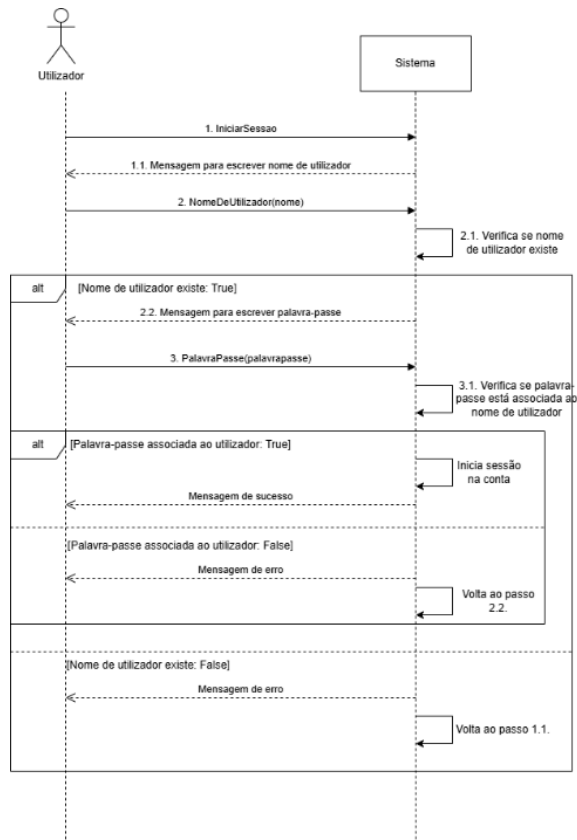


figura 22

UC – Partilhar Música

Na figura 23 vemos o fluxo de execução para partilha de músicas. Como o fluxo será parecido para as músicas e para as listas de reprodução, usamos o exemplo da uc de partilhar música.

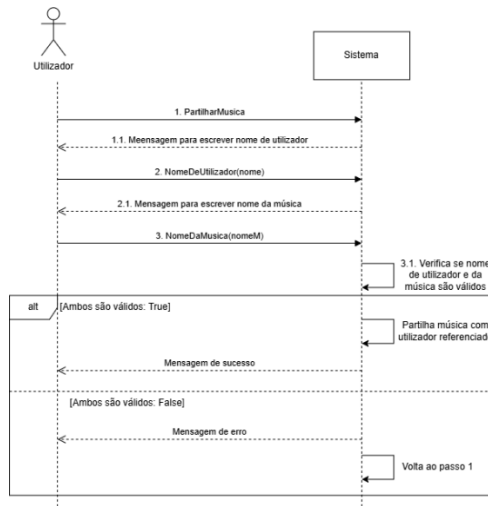


figura 23

UC – Reproduzir Música

Com o diagrama apresentado na figura 24 vemos como o utilizador pode reproduzir uma música através do programa.

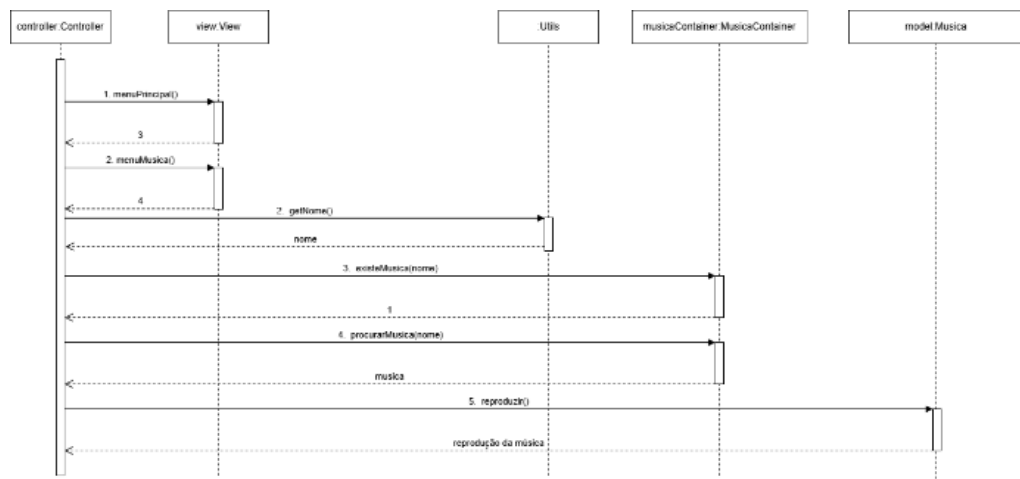


figura 24

UC – Pesquisar Música

O diagrama apresentado na figura 25 representa o processo de pesquisa de uma música.

UC - Pesquisar música

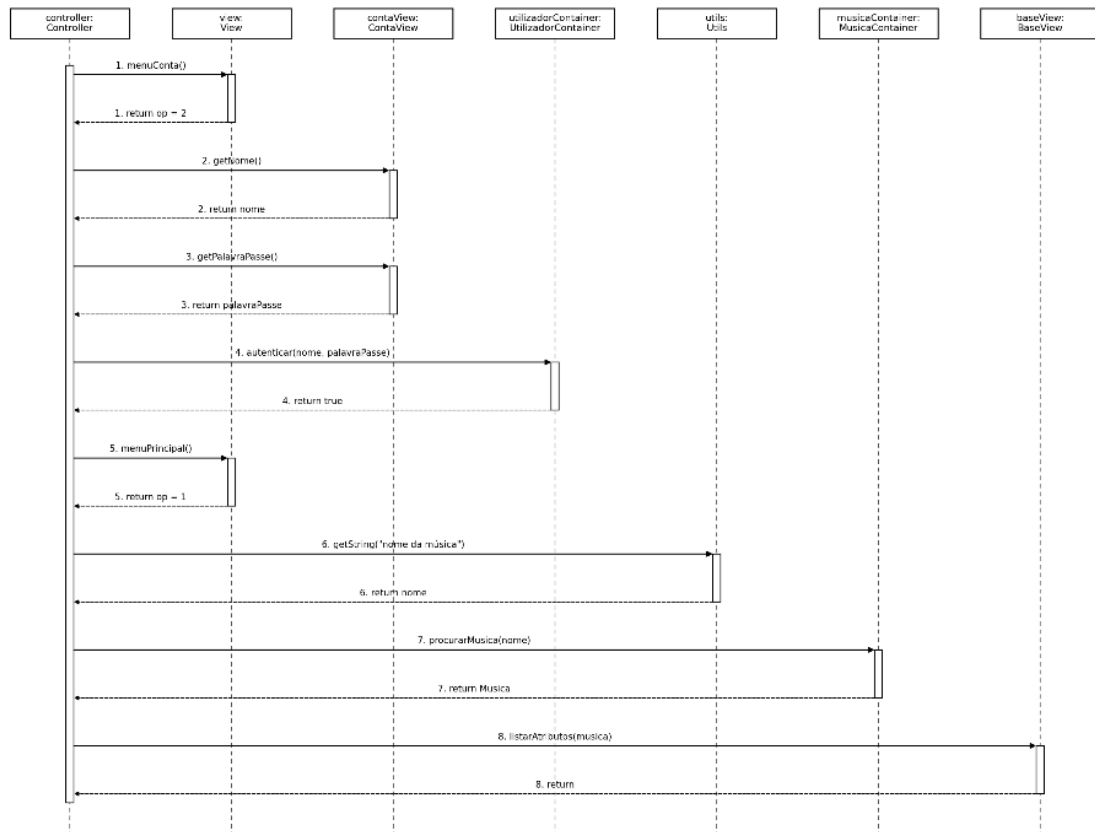


figura 25

UC – Procurar Álbum

Da mesma forma, o processo de pesquisar álbuns, editoras e artistas é semelhante, então usamos o exemplo do diagrama da figura 26.

UC2 – Pesquisar álbum

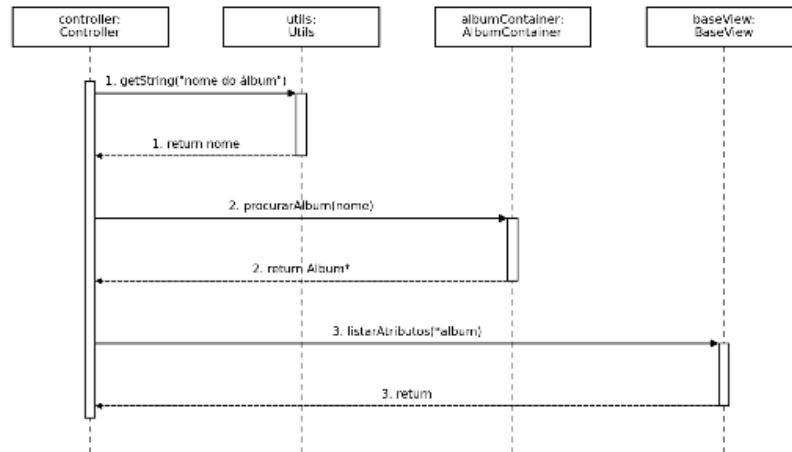


figura 26

UCs – Adicionar Música à Lista de Reprodução/Remover Música da Lista de Reprodução

Na figura 27 e figura 28 estão representados os processos para a adição e remoção de músicas às listas de reprodução.

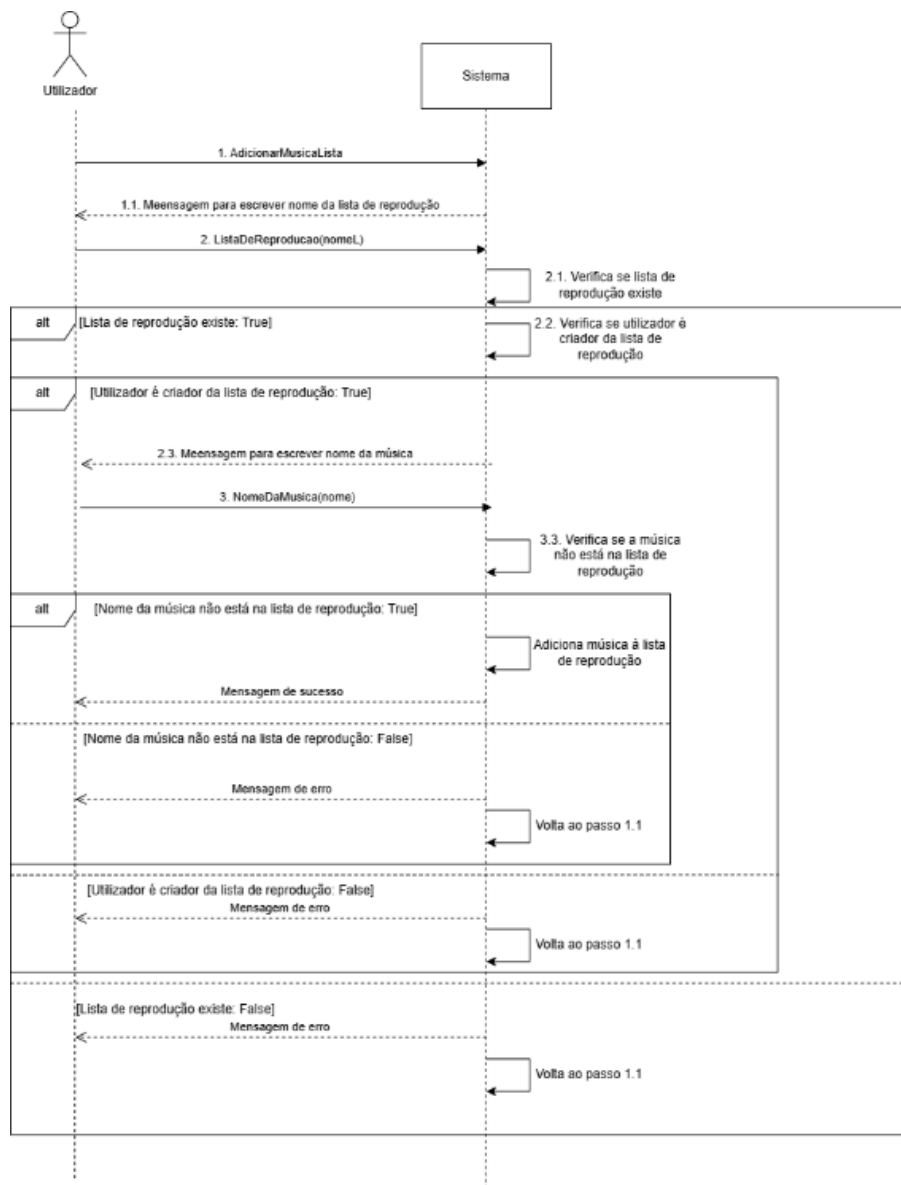


figura 27

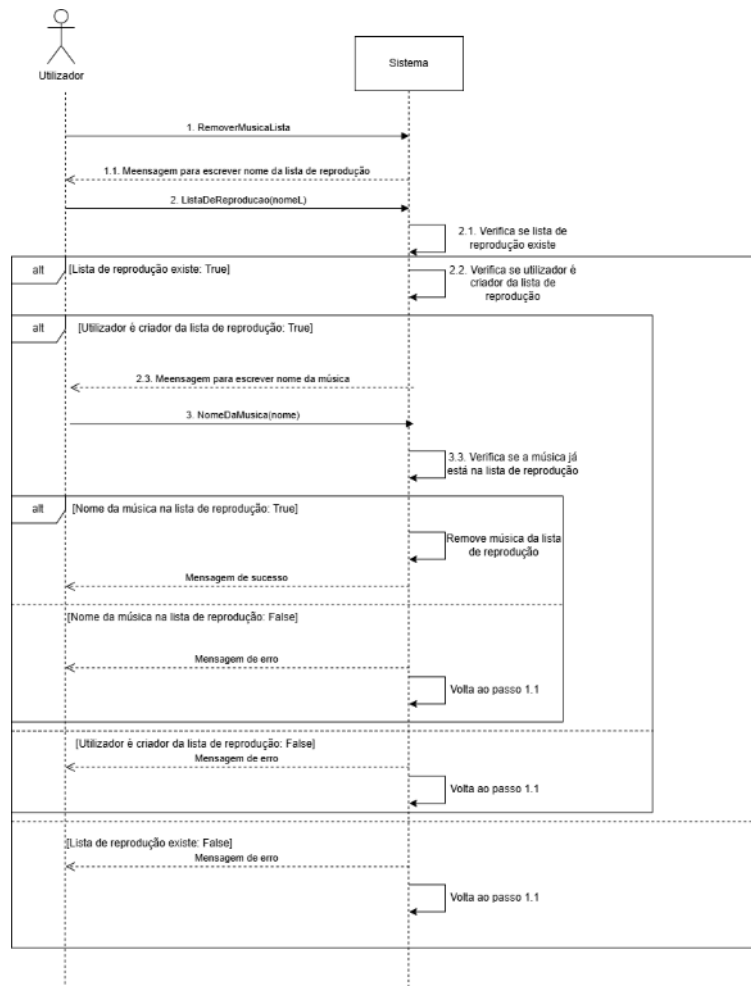


figura 28

UCs – Adicionar/Remover Músicas

Nos seguintes diagramas (figura 29 e figura 30) estão representados os fluxos para remover e adicionar músicas ao programa. Como o processo de adicionar e remover é semelhante para músicas, artistas, editoras, álbuns e listas de reprodução, foram usados estes diagramas como exemplo.

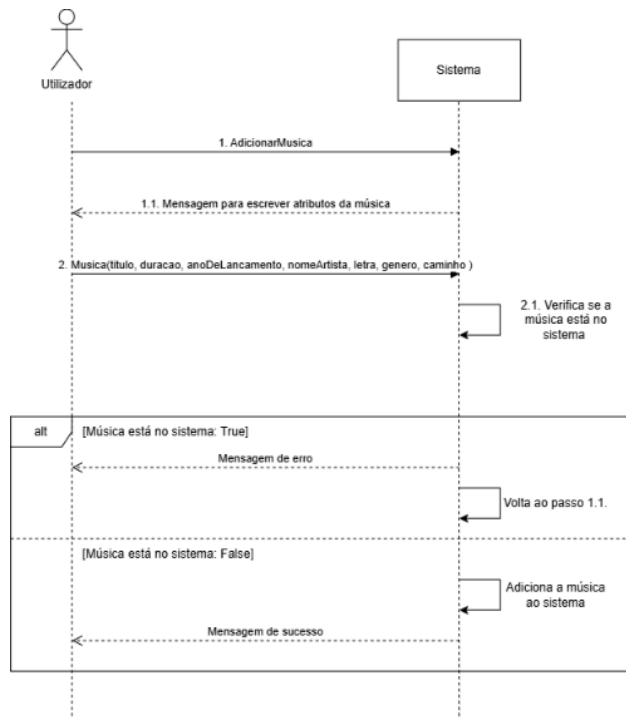


figura 29

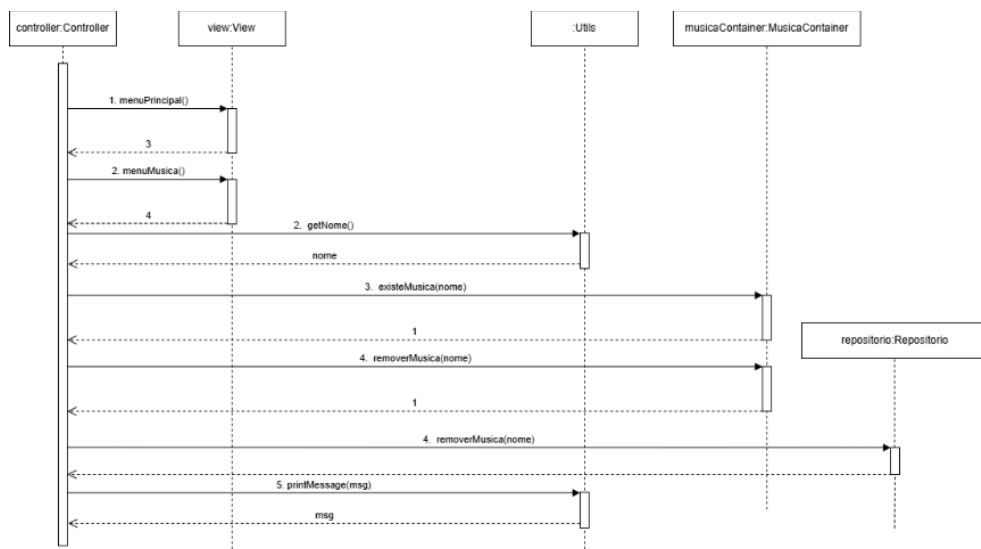


figura 30

UC – Partilhar Música

Através do fluxo apresentado na figura 31, o utilizador pode partilhar músicas.

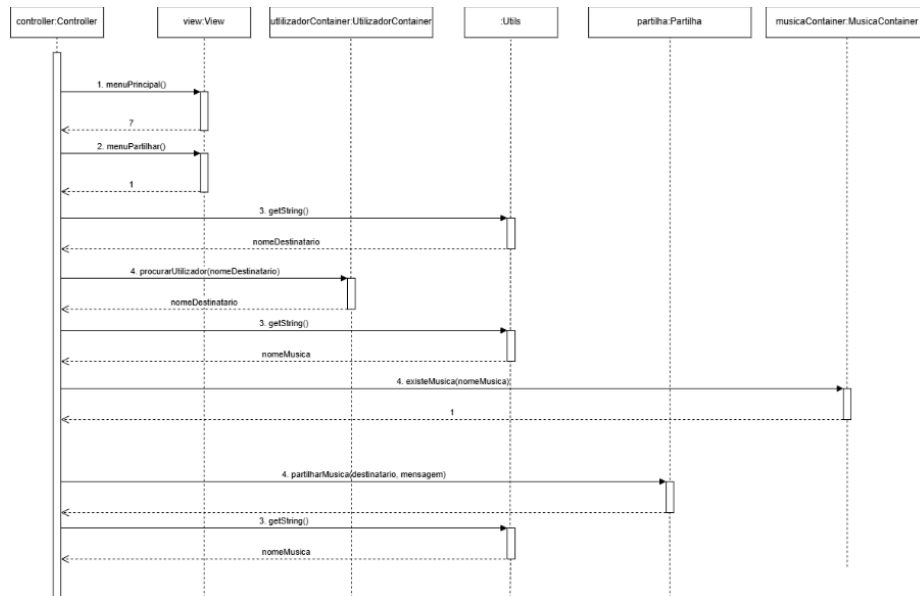


figura 31

UC – Ver Partilhas

O diagrama apresentado na figura 32 revela como o utilizador pode ver as partilhas.

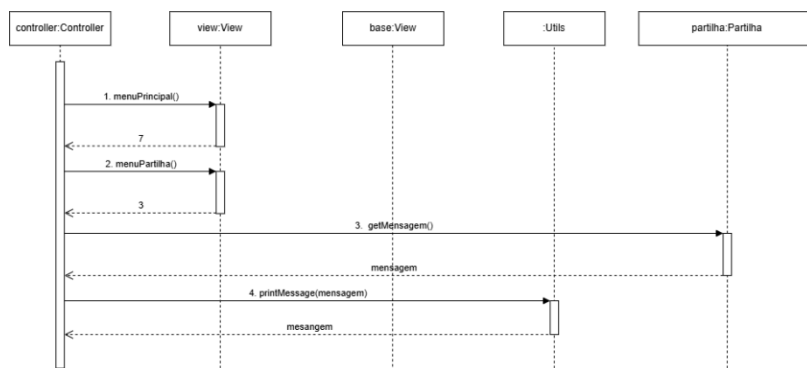


figura 32

UC – Sair da Conta

Por último, está representado no diagrama da figura 33 o fluxo do programa para o utilizador sair da conta.

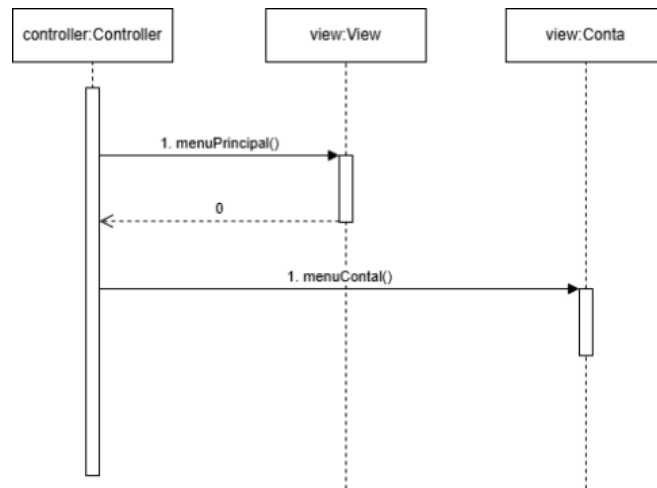


figura 33

4.8 Interface do Utilizador

A interface do utilizador é composta pelo conjunto de menus que estão disponíveis ao utilizador e as relações entre eles. A figura 34 apresenta uma representação dos menus que serão apresentados ao utilizador ao longo da interação com o programa. Observa-se também, na figura, os números que devem ser introduzidos na linha de comandos para permitir a navegação entre os menus.

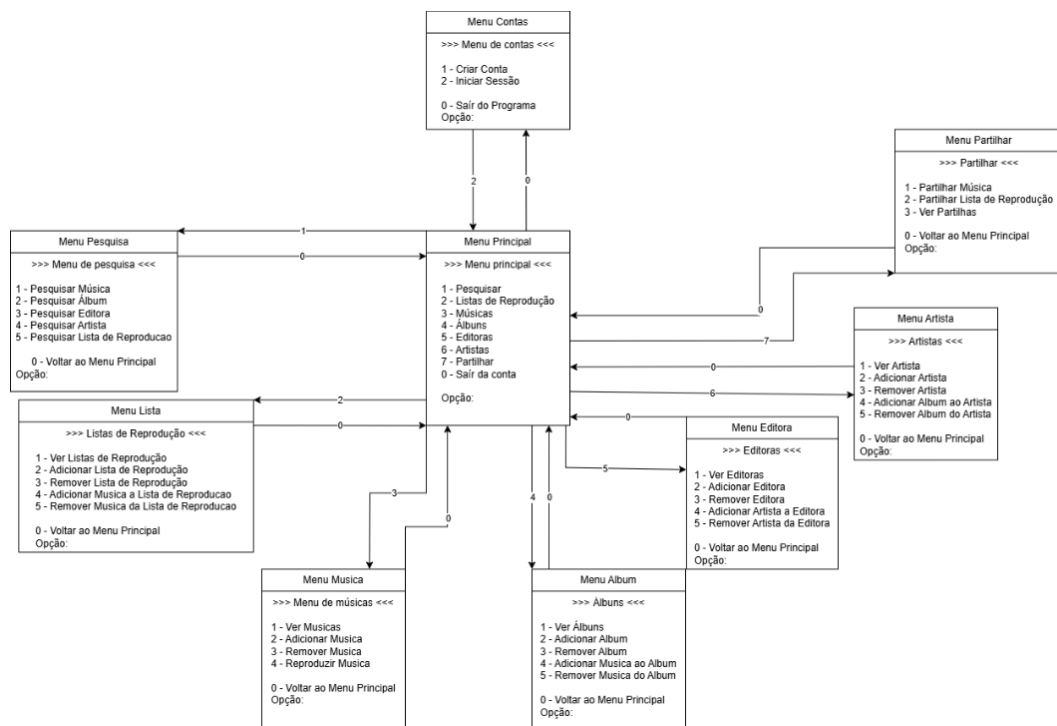


figura 34

5 Implementação

A implementação é a fase do desenvolvimento de software em que tudo que foi definido até agora é colocado em prática. Nesta fase desenvolvemos o programa de acordo com as especificações, desenho e requisitos que foram explicitados até aqui.

No processo de desenvolvimento utilizamos apenas o IDE (Integrated Development Environment) chamado CLion, a linguagem de programação escolhida foi o C++, e para a persistência de dados utilizamos o formato de notação de dados JSON.

De modo a permitir o controlo de versões neste projeto em grupo, recorreremos à tecnologia Git e, por extensão, à plataforma web GitHub, onde está localizado o repositório central.

5.1 Código

Cada classe está dividida em dois ficheiros distintos: um tem a terminação .cpp e o outro .h. O ficheiro .h, denominado header file, é onde se define uma classe, enquanto que no .cpp se realiza a implementação da mesma.

A estrutura de ficheiros do nosso programa pode ser vista na captura de ecrã que se segue:

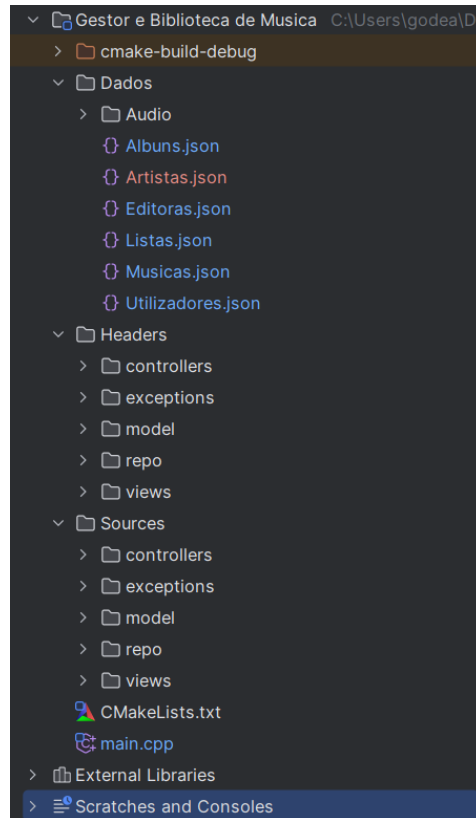


figura 35

Observa-se que os ficheiros .cpp estão armazenados em Sources e os .h em Headers. No diretório Dados estão armazenados os ficheiros .json, onde são guardadas as informações necessárias, e a pasta audio é para os ficheiros áudio.

5.2 Classes

O programa como um todo tem inúmeras classes, portanto, para não criar um relatório extremamente extenso, nesta secção abordamos a implementação de apenas uma das

classes do Gestor e Biblioteca de Música, nomeadamente as classes Musica e MusicaContainer.

A classe Musica representa a entidade central do programa: uma música individual. Segundo o princípio *Information Expert* do GRASP, é ela que detém todos os dados de uma música e a única responsável por geri-los. Cada objeto guarda os atributos nome, duracao, anoDeLancamento, letra, genero, caminho e nomeArtista.

A classe disponibiliza dois construtores: um vazio (usado, por exemplo, no carregamento de dados) e outro com parâmetros, que inicializa todos os seus atributos.

O acesso aos dados é feito através de *getters* (getNome(), getDuracao(), getLetra(), etc.), todos declarados como const por não alterarem o estado do objeto.

Além de armazenar a informação, a Musica sabe reproduzir-se a si própria, através do método reproduzir(). Este recorre à função ShellExecute da API do Windows para abrir o ficheiro.

Nas duas capturas apresentadas a seguir lado a lado, observamos a implementação do ficheiro Musica.cpp.

```
#include "Musica.h"
#include <string>
#include <iostream>
#include <sstream>
#include "windows.h"
#include <shellapi.h>

Musica::Musica(){}

Musica::Musica(
    std::string nome,
    int duracao,
    int anoDeLancamento,
    std::string letra,
    std::string genero,
    std::string caminho,
    std::string nomeArtista){

    this->nome = nome;
    this->duracao = duracao;
    this->anoDeLancamento = anoDeLancamento;
    this->letra = letra;
    this->genero = genero;
    this->caminho = caminho;
    this->nomeArtista = nomeArtista;
};

Musica::~Musica(){}

std::string Musica::getLetra() const {
    return letra;
};

int Musica::getDuracao() const {
    return duracao;
};

std::string Musica::getNome() const {
    return nome;
};

void Musica::reproduzir() const
{
    ShellExecute(
        NULL,
        "open",
        caminho.c_str(),
        NULL,
        NULL,
        SW_SHOWNORMAL);
};

int Musica::getAnoDeLancamento() const {
    return anoDeLancamento;
};

std::string Musica::getCaminho() const {
    return caminho;
};

std::string Musica::getGenero() const {
    return genero;
};

std::string Musica::getNomeArtista() const {
    return nomeArtista;
};
```

figura 36

A classe MusicaContainer é responsável por gerir a *coleção* de todas as músicas existentes no programa, concentrando num único sítio todas as operações sobre esse

conjunto. A coleção é guardada num `std::vector<Musica>` declarado como `static`. O `MusicaContainer` oferece as operações típicas de gestão da coleção:

- `adicionarMusica(const Musica&);`
- `removerMusica(const std::string& nome);`
- `existeMusica(const std::string& nome);`
- `procurarMusica(const std::string& nome);`
- `getMusicas();`
- `reproduzirMusica(std::string n)`

A implementação do `MusicaContainer` apresenta-se na captura de ecrã que se segue:

```

1 #include "MusicaContainer.h"
2 #include "Musica.h"
3 #include "../Headers/repo/Repositorio.h"
4 #include <string>
5
6
7
8 std::vector<Musica> MusicaContainer::musicas;
9
10 MusicaContainer::MusicaContainer() {}
11
12 void MusicaContainer::adicionarMusica(const Musica& musica) {
13     musicas.push_back(musica);
14 }
15
16 bool MusicaContainer::removerMusica(const std::string& nome) {
17     for (auto it = musicas.begin(); it != musicas.end(); ++it) {
18         if (it->getNome() == nome) {
19             musicas.erase(it);
20             return true;
21         }
22     }
23     return false;
24 }
25
26 bool MusicaContainer::existeMusica(const std::string& nome) {
27     for (auto& m : musicas) {
28         if (m.getNome() == nome) return true;
29     }
30     return false;
31 }
32
33 Musica* MusicaContainer::procurarMusica(const std::string& nome) {
34     for (auto& m : musicas) {
35         if (m.getNome() == nome) return &m;
36     }
37     return nullptr;
38 }
39
40 const std::vector<Musica>& MusicaContainer::getMusicas() const {
41     return musicas;
42 }
43
44 void MusicaContainer::reproduzirMusica(std::string n) {
45     for (int i = 0; i < musicas.size(); i++) {
46         if (musicas[i].getNome() == n) {
47             musicas[i].reproduzir();
48         }
49     }
50 }
51
52 };

```

figura 37

A divisão das entidades numa classe para gerir os seus dados e outra que serve de container, usada para gerir uma coletânea de objetos, permite aumentar a coesão do programa, fazendo com que as classes tenham responsabilidades mais granulares.

6 Testagem

Depois de implementar na totalidade o programa, realizamos alguns testes de certas funções, recorrendo à Google Test Framework, que é uma biblioteca especializada na realização de testes unitários na linguagem de programação C++. Mais especificamente, foram testadas as funções do MusicaContainer: adicionar, existe, procurar e remover, bem como o construtor e os getters da classe Musica.

Resumimos os resultados dos testes concluídos na seguinte tabela:

Teste	Resultado
Construtor + getters da classe Musica	100% sucesso
MusicaContainer.existeMusica()	100% sucesso
MusicaContainer.adicionarMusica()	100% sucesso
MusicaContainer.procurarMusica()	100% sucesso
MusicaContainer.removerMusica()	100% sucesso
MusicaContainer.getMusicas()	100% sucesso

Os testes foram implementados no ficheiro test_musica.cpp. O código presente nesse ficheiro é mostrado nas duas imagens a seguir:

```

#include <gtest/gtest.h>
#include "Musica.h"
#include "MusicaContainer.h"

// Testes da CLASSE Musica (construtor + getters)
TEST(MusicaTest, ConstrutorGuardaTodosOsAtributos) {
    Musica m("nome_musical", 354, 1975,
            "letra_musical",
            "genero_musical", "/audio/musical.mp3", "artista_musical");

    EXPECT_EQ(m.getNome(), "nome_musical");
    EXPECT_EQ(m.getDuracao(), 354);
    EXPECT_EQ(m.getAnoDeLancamento(), 1975);
    EXPECT_EQ(m.getLetra(), "letra_musical");
    EXPECT_EQ(m.getGenero(), "genero_musical");
    EXPECT_EQ(m.getCaminho(), "/audio/musical.mp3");
    EXPECT_EQ(m.getNomeArtista(), "artista_musical");
}

// Testes do MusicaContainer (adicionar / existe / procurar / remover)
class MusicaContainerTest : public ::testing::Test {
protected:
    MusicaContainer container;

    void SetUp() override {
        MusicaContainer::musicas.clear();
    }

    Musica criarMusica(const std::string& nome) {
        return Musica(nome, 200, 2020, "letra", "Pop", "/p.mp3", "Artista");
    }
};

TEST_F(MusicaContainerTest, AdicionarTornaMusicaExistente) {
    container.adicionarMusica(criarMusica("nome_musical"));

    EXPECT_TRUE(container.existeMusica("nome_musical"));
    EXPECT_EQ(container.getMusicas().size(), 1u);
}

```

figura 38

```

TEST_F(MusicaContainerTest, ExisteEhFalsoQuandoNaoFoiAdicionada) {
    EXPECT_FALSE(container.existeMusica("nome_musica_inexistente"));
}

TEST_F(MusicaContainerTest, ProcurarDevolveAMusicaCorreta) {
    container.adicionarMusica(criarMusica("nome_musica1"));

    Musica* m = container.procurarMusica("nome_musica1");
    ASSERT_NE(m, nullptr); // se for nullptr, nao vale a pena
    EXPECT_EQ(m->getNome(), "nome_musica1");
}

TEST_F(MusicaContainerTest, ProcurarDevolveNullptrSeNaoExiste) {
    EXPECT_EQ(container.procurarMusica("nome_musica_inexistente"), nullptr);
}

TEST_F(MusicaContainerTest, RemoverDevolveTrueEApaga) {
    container.adicionarMusica(criarMusica("nome_musica1"));

    EXPECT_TRUE(container.removerMusica("nome_musica1"));
    EXPECT_FALSE(container.existeMusica("nome_musica1"));
    EXPECT_EQ(container.getMusicas().size(), 0u);
}

TEST_F(MusicaContainerTest, RemoverDevolveFalseSeNaoExiste) {
    EXPECT_FALSE(container.removerMusica("nome_musica_inexistente"));
}

TEST_F(MusicaContainerTest, GetMusicasContaCorretamente) {
    container.adicionarMusica(criarMusica("nome_musica1"));
    container.adicionarMusica(criarMusica("nome_musica2"));
    container.adicionarMusica(criarMusica("nome_musica3"));

    EXPECT_EQ(container.getMusicas().size(), 3u);
}

```

figura 39

Conclusão

Ao longo das três iterações deste projeto, partimos de um modelo de domínio e de um conjunto de casos de uso para chegar a uma aplicação de linha de comandos completamente funcional, capaz de gerir músicas, álbuns, artistas, editoras e listas de reprodução, bem como de partilhar conteúdos entre utilizadores na mesma máquina. Este processo permitiu-nos perceber, e interiorizar profundamente nas nossas almas, conceitos fundamentais da engenharia de software e da programação orientada a

objetos. Em síntese, o projeto cumpriu todos os objetivos a que se propôs e constituiu uma oportunidade valiosa de aprendizagem.